A Seed That Grows by Proving Its Own Changes: One Agent, One Carried State, Growth Across Five Areas and Three Seeds

Maureese Williams

October 11, 2026

Abstract

This paper reports a seed: one agent with one carried state that improves itself across areas by keeping only the changes it can prove. The state is a language of atoms its hypothesis catalog is built from, the experience it earned in every world it has lived, a world and self model and a prior per area fitted from that experience, and a ledger. The seed runs four operations on itself. It lives in an area and earns atoms by solving tasks under a fixed scan budget with no outcome feedback. It reflects, refitting an area's model and prior from everything earned there, choosing the model's size by its own held-out likelihood. It extends its language by inventing atoms from the expression grammar for the worlds it could not solve, keeping a candidate only if it recurs across unsolved tasks. It prunes invented atoms that never earned their keep. No change is kept until it solves more of a held-out set, drawn fresh for every life, than the carried state does, and a change to the shared language must leave every other area the seed has lived in no worse. On a held-out set never used for any proof, the seed on development seed 644101 went from 1,299 to 2,072 tasks of 7,200 in mechanism worlds over ten lives, from 181 to 338 of 2,400 in a drawn random partition of atoms over four lives, from 375 to 1,913 of 4,800 in metric worlds whose structure is behavioural over four lives, and from 215 to 296 and from 224 to 422 of 2,400 in two further drawn partitions, with four and two groups per world, over three and two lives. Fresh seeds on 643101 and 645101 went from 1,372 to 2,006 and from 1,159 to 2,048 in three mechanism lives. Models were kept on nearly every life, 166 invented atoms stand in the language, 56 of them reproducing atoms of the two families withheld from the seed, and three prunings were kept. The three kinds of growth the programme had reproduced separately, growth within a life, growth of acquisition through a learned representation, and growth of reach by invention, run here in one artifact for the first time. The evidence is development evidence on three opened seeds with no predeclared gate. The learner code is fixed; what the seed improves is the state that learner runs on.

1 Introduction

The goal is a seed that improves in as many areas as possible, with the work focused on the seed itself and every proven success carried forward. Before this work the programme had reproduced three kinds of growth, in three separate codebases, with nothing carried between runs except files written by hand.

The first is growth within a life. An agent that keeps the atoms of whatever it submits, right or wrong, raises its own solve rate four to ten times across a hundred tasks at every budget from 128 to 4,096 scans, and being told whether it was right buys nothing. The second is growth of acquisition through a learned representation. A grouping of atoms learned without labels from 1,600 worlds

solves 6,993 and 8,077 more tasks over 18 budget cells than the same agent on a grouping from 400 worlds, confirmed on predeclared seeds and audited, and the same gain was seen, with a safety failure at two budgets, when the grouping was learned from the agent's own charged picks. The third is growth of reach by invention. In the string DSL, a system that synthesises primitives from failures solved 20 of 24 tasks where its fixed grammar solved 5, and a self-editing grammar solved 18 where the fixed one solved 12, on two seeds each, one generation deep.

None of these was recursive in the programme's sense, a later improver producing a better child improver. Every attempt at that came back null. What they had in common was a state that grew with experience and a learner that did not. The step taken here is to put the three kinds of growth into one artifact whose state is the thing that grows, and to make the artifact decide for itself which changes to its state to keep. The standard for keeping a change is the only one the programme has found to hold up: it must solve more held-out tasks than the state before it, measured on worlds it never trained on, drawn fresh each time, with the measurement charged to the seed. A seed built this way cannot claim a gain it did not earn, and it cannot keep a change that helped in one area by hurting another, because a change to its shared state is checked everywhere it has lived.

The question this paper answers is narrow. Does one such seed, started with a language that lacks two of the nine atom families and no model of any world, grow across several areas with different structure, and does the growth hold on worlds it never proved anything on? It does, on three development seeds, and the paper reports how much, what it kept, what it invented, where it failed, and what the result does not establish.

2 The seed

The seed is one artifact: a state file and four operations that read and rewrite it. Nothing else persists between runs, and nothing in the operations is tuned to an area.

State. The language is the list of atoms the seed's hypothesis catalog is built from; the catalog itself, every atom, every cross-family pair under three boolean operators and every complement reduced to one row per distinct behaviour, is rebuilt from the language whenever it changes. The experience is one record per world lived: the area, the atoms behind the seed's confident picks, and the atoms behind all its picks, by atom name, so that the records survive the language changing under them. Per area the seed holds a world and self model, a grouping of atoms with how often each group set occurs and how often it finds each atom, and a flag saying whether to build a pair prior from that area's experience. It also holds the atoms it invented and the life they entered, a ledger of every change it tried with its proof, and the resource it has spent on living, proving and inventing.

Live. In an area, the seed draws fresh worlds and lives each as a stream of 100 tasks at a fixed scan budget. Its solver is the earned loop: an ordering of the catalog, a scan of the first budget rows, the smallest consistent candidate submitted, its atoms kept as evidence. When the seed has a model for the area, the ordering reads it: a posterior over which group sets the current world uses, updated from the atoms the seed has already earned there, lifts the rows those groups build. When it has a prior, the ordering starts from pair co-occurrence in its own earlier worlds, with half the budget reserved for the order it would use with no prior at all. With neither it scans in catalog order. Every pick is recorded with the number of consistent hypotheses in the window, and a pick with exactly one is a confident pick.

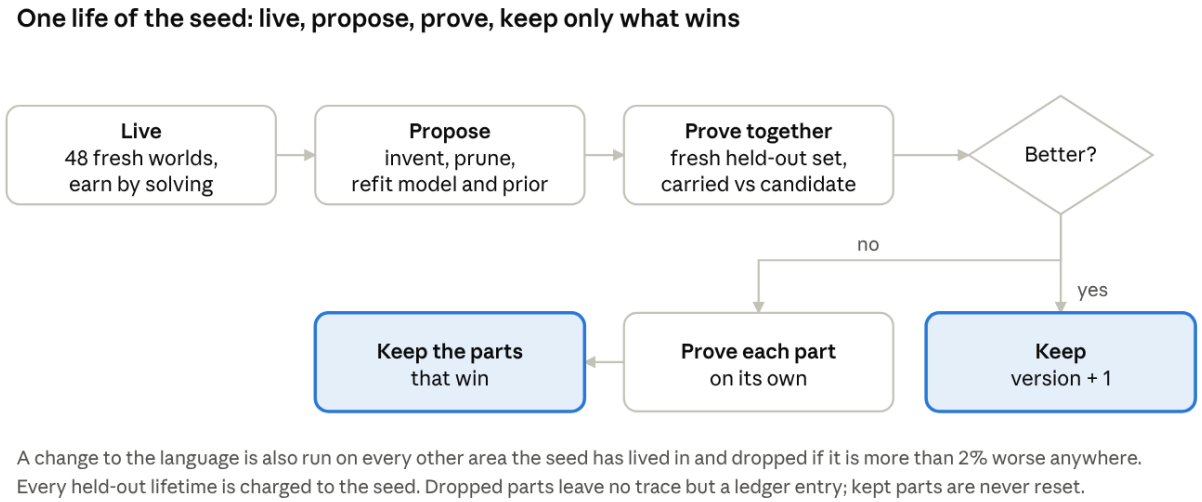


Figure 1: One life of the seed. Live, propose every change at once, prove the combination on a fresh held-out set, keep it if it wins, otherwise prove each part on its own. A change to the shared language is also run on every other area the seed has lived in and dropped if it is more than 2% worse anywhere. Every held-out lifetime is charged to the seed.

Reflect. From the area's confident picks the seed fits its world and self model by expectation-maximization, as in the companion paper: group labels per atom, group-set frequencies, a discovery rate and an import rate per atom, an atom being filed in no group when the agent would almost never find it in a world. It fits group counts of 6, 7 and 8 and 2, 3 or 4 groups per world and keeps the pair with the best likelihood on its last fifth of worlds. The new model and a prior from the area's experience form one candidate.

Extend. For the top third of the life's worlds by tasks left unsolved, and only worlds with at least 15 unsolved, the seed enumerates depth-one expressions from the ord_expr grammar, 3,456 candidates distinct up to complement, in breadth-first order, and tests each against the constraints of the unsolved tasks, alone, complemented, or joined to one atom it earned in that world by and, or or xor. A candidate is kept when it fits at least two uncovered tasks, because an atom the world is built from recurs across its mechanisms and a chance fit does not. Up to 40 new atoms per life form the candidate language.

Prune. An invented atom that has not appeared in a confident pick in any world two or more lives after it entered is proposed for removal.

Prove. Every candidate is run against the carried state on a held-out set drawn fresh for the life. The combination of all proposals is tried first; if it loses, each part alone. Only a candidate that solves strictly more is kept, the version counter advances, and the ledger records the change, its score and its cost. A candidate that changes the language, which every area shares, is also run on a smaller held-out set of each other area the seed has lived in, and dropped if it is more than 2% worse in any of them.

A life ends with at most three kept changes and at least one ledger entry (Figure 1). Nothing kept is ever undone except by a later proven prune.

3 Areas

All areas share one hypothesis catalog, 166,596 behaviourally distinct character predicates built from one or two atoms out of 313 atoms in 9 families, and one task form: a world is 14 atoms, a task shows 8 visible examples of one of the world's predicates and holds 16 hidden, and the agent scans the catalog in its own order and submits the smallest consistent predicate. What differs between areas is the latent structure that decides which atoms share a world. The seed is never told that structure, which family an atom belongs to, or which area a world came from. The pool names below exist for the ledger and for scoring.

Mechanism worlds (area `mech`). A world is drawn from three of seven families, and its atoms from their pooled atoms. The seed's starting language lacks the two remaining families, popcount and xormod, so a world that uses one of them holds mechanisms the seed cannot reach by scanning at any budget. A fifth of the viable family triples are kept out of the seed's lives as untrained combinations. Lives draw three quarters of their worlds from trained triples and a quarter from triples with an unfamiliar family. Held-out sets hold equal numbers of trained, untrained and unfamiliar worlds.

Drawn partitions (areas `drawn7`, `drawn11`, `drawn13`). Every atom of every family is assigned at random to one of a few groups, a world picks some groups and draws its atoms from their union, and each atom slips to a uniformly random atom with some probability. The partition cuts across families, so nothing the seed learned about mechanism worlds applies here except its language. The three areas differ in structure: 9 groups with 3 per world and slip 0.15, 12 groups with 4 per world and no slip, and 6 groups with 2 per world and slip 0.15. The seed does not know how many groups a world uses; it chooses that by held-out likelihood when it reflects.

Metric worlds (area `metric777001`). Each atom has a behavioural descriptor, the normalised Fourier and Walsh spectra of its truth mask over the alphabet. A sparse weighting over descriptor dimensions is drawn once and shown to nobody, and a world is a centre atom and the fourteen atoms nearest it under that weighting. Here the structure is behavioural rather than syntactic, and a quarter of each life's worlds are centred on an atom of the two families the seed's language lacks.

What the seed carries between areas is its language and its experience. What it fits per area is the model and the prior, because a grouping that explains one area's worlds says nothing about another's. The ledger and the proofs are shared, and so is the resource.

4 Protocol

A life is 48 worlds of 100 tasks each at a budget of 1,024 scans per task. A run is a number of lives in one area; the seed's state is saved after every operation, so lives in different areas can be interleaved and any run resumed. The development seed 644101 (seed `s1`) lived ten lives in mechanism worlds, four in `drawn7`, four in `metric777001`, three in `drawn11` and two in `drawn13`. Seeds `s2` and `s3`, on 643101 and 645101, lived three mechanism lives each.

After each life the seed proposes every change it has: the atoms it invented for the worlds it could not solve, the invented atoms that never earned their keep, and a refit of the area's model and prior. It proves the combination first, and if the combination fails, each part on its own. A proof runs the carried state and the candidate on a held-out set of the area, 24 worlds per pool, drawn fresh for that life, so no set is reused by a later proof. A candidate is kept when it solves

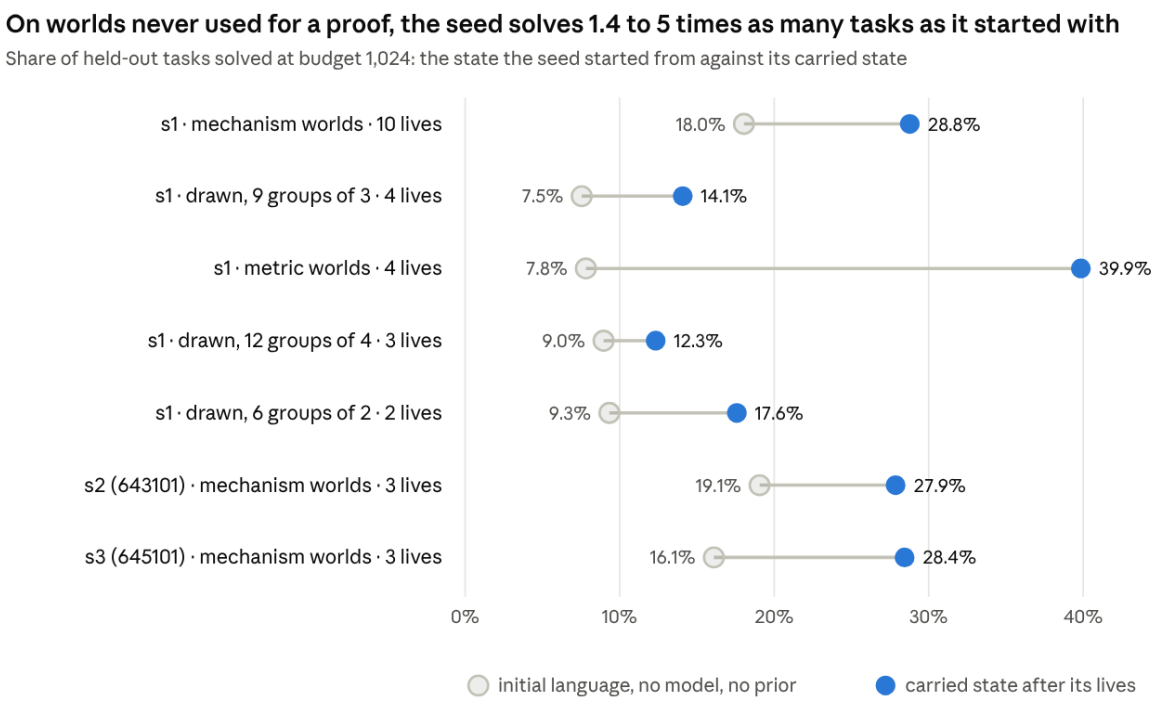


Figure 2: Share of held-out tasks solved at budget 1,024 on sets no proof used: the state the seed started from (open) against its carried state (filled), per area and seed.

strictly more held-out tasks than the carried state. A candidate that changes the language is also run on a 12-per-pool held-out set of every other area the seed has lived in, and is dropped if it falls more than 2% below the carried state anywhere. Every held-out lifetime is charged to the seed's resource, as are the invention tests, so the ledger records the full cost of growing, not only of living.

The seed never sees a family label, a true world, an area name as anything but a key for its own model, or whether a submission was correct. The only signals it reads are its own picks, the number of consistent hypotheses it found before each pick (a pick made with exactly one consistent hypothesis in the window is correct 90% of the time, and only such picks enter its model), the tasks on which it found no consistent candidate, and its held-out solve counts. Labels enter only when the results are scored, to say which pool a world came from and which family an invented atom reproduces.

A final check that the seed never performs: after a run, its carried state and the state it started from (its initial language, no model, no prior) are run on one more held-out set of each area, drawn with a salt no proof ever used. Those are the numbers reported as growth. The per-life held-out numbers in the ledger are the ones the seed itself read.

5 Results

The seed grew in every area it entered and on every seed it was started from (Figure 2, Table 1). In mechanism worlds `s1` went from 1,299 to 2,072 of 7,200 held-out tasks over ten lives, with trained worlds from 26 to 50 tasks per world and worlds of an unfamiliar family from 4 to 19, while untrained combinations fell from 24 to 18. `s2` and `s3`, started fresh on other seeds, went from 1,372 to 2,006 and from 1,159 to 2,048 in three lives. In the drawn partition `s1` went from 181 to 338 of 2,400,

Table 1: Growth on held-out sets never used for a proof, tasks solved, 24 worlds per pool at budget 1,024.

Seed	Area (lives)	Start	Carried	Tasks
s1	mechanism worlds (10)	1,299	2,072	7,200
s1	drawn7, 9 groups of 3 (4)	181	338	2,400
s1	metric worlds (4)	375	1,913	4,800
s1	drawn11, 12 groups of 4 (3)	215	296	2,400
s1	drawn13, 6 groups of 2 (2)	224	422	2,400
s2 (643101)	mechanism worlds (3)	1,372	2,006	7,200
s3 (645101)	mechanism worlds (3)	1,159	2,048	7,200

and in metric worlds from 375 to 1,913 of 4,800, where a model fitted from its first 48 worlds was alone worth 975 tasks on that life's held-out set. In the two later drawn partitions, with four and two groups per world, it went from 215 to 296 and from 224 to 422 of 2,400 in three and two lives, keeping one model in each; for the first it chose four groups per world, which is that area's true structure, and for the second also four, where the truth is two.

What it kept. Over 24 versions `s1` kept 15 models, 6 language extensions and 3 prunings, and dropped 16 extensions and 5 models. A model was kept on every mechanism life from 0 to 6, with gains on the seed's own held-out sets of 417, 118, 13, 133, 98, 2 and 11, then on lives 8 and 9 the gains were gone. In the drawn partition it kept a model on its first three lives and a pruning on the first and fourth; in metric worlds a model on three lives of four, an extension on the second and a pruning on the fourth. On `s2` and `s3` a model was kept on all three lives (475, 50, 59 and 412, 74, 118) and every extension was dropped. The model line is the reliable one; invention pays off only sometimes, and when it does the gain is smaller.

What it invented. At version 17 the language held 151 invented atoms: 25 reproduced true popcount atoms and 28 true xormod atoms, the two families withheld from the seed, and 98 were atoms of neither family nor any other, behaviourally new. By version 24 the count was 166, after one more extension and one more pruning: 27 popcount, 29 xormod and 110 behaviourally new. Invented atoms entered only through a proof, and the seed's own prune then removed 23, 26 and 25 of them in three proven steps when they had never been earned. On mechanism worlds with an unfamiliar family, where the seed's starting language cannot reach a third of the mechanisms, invention is the only route to the growth from 4 to 19 tasks per world.

What it cost. By version 24 `s1` had spent 113.0 million scans living, 216.3 million scans proving, and 737,143 invention tests. Proving cost nearly twice living, because every life proved up to three candidates on 72 worlds each against a freshly scored carried state, and the cross-area checks added to that. That ratio is the first thing to cut; it does not change what was kept.

What the seed read against what held. The per-life held-out totals the seed used to decide were, for the first four mechanism lives on one fixed set, 1,291, 1,708, 1,722, 1,840, 1,853, 1,898 and 2,031, and on fresh sets thereafter 2,129, 2,131, 2,142, 2,195, 2,234 and 2,089. The fresh set used for Table 1, never seen by any proof, gave 2,072. The seed's own reading at the end was 2,089 on that

life's set. The numbers it decides by and the numbers that hold are within a few percent of each other.

6 Interference between areas

The three-area check exposed a flaw in the first version of the seed. After four lives in metric worlds, `s1`'s score on the drawn partition's untouched held-out set fell from 376 to 338, a 10% loss in an area it had not entered since. The cause was the shared language. The metric lives had kept one extension of 40 atoms and one pruning of 25, each proven on the metric area alone, and a language that grew and shrank to suit metric worlds changed the catalog the drawn partition's model was fitted over. Each change was a proven success where it was proven and a regression where it was not.

The fix is in the proof rule, not in the areas. A candidate that changes the language is now also run on a held-out set of every other area the seed has lived in, 12 worlds per pool, and is dropped if it falls more than 2% below the carried state in any of them. The carried scores of those areas are recomputed after any kept shared change, so a later proof never compares against a stale number. The cost is more proof lifetimes per language candidate, which is why the combination of all proposals is proven first and the parts only if it fails.

The rule fired twice as soon as it existed. In `drawn11` a combination of invented atoms and a model gained 165 to 344 on that area's held-out set and was dropped because it cost mechanism worlds 8% and `drawn7` 12%; the model alone was then kept, at 417. In `drawn13` a combination worth 150 was dropped for costing `drawn11` 23%. Each would have been a proven success a day earlier.

Two related changes went in at the same time. The seed now chooses how many groups a world uses, 2, 3 or 4, by held-out likelihood when it reflects, so that it can model the drawn areas whose worlds use two or four groups without being told. And a pool that dies from the machine's shared memory is retried with fewer workers, because the one stall in the record was a worker pool whose processes could not start while other jobs held the machine's commit, and a seed that cannot survive its host is not much of a seed.

The lesson is the same one the programme has met before in other forms: a change is proven only against what it was tested on, and a state shared across areas must be tested across areas. Carrying forward proven successes means proving them wherever they will be carried.

7 What this is and is not

It is one artifact that improves its own state in five areas of different structure and on three seeds, keeping only changes it proved on worlds it never trained on, with the proofs charged. The three kinds of growth the programme had reproduced separately run in it together, and each shows up in the ledger: the earned loop inside every life, the model and prior that are kept nearly every life and account for most of the gain, and the invented atoms that let it reach families its starting language could not express. Growth in one area did not require starting over in another; the language carried, and in the metric area a model fitted from 48 worlds was worth 975 tasks at once.

It is development evidence. The seeds were opened, no gate was predeclared, no forecast was committed, and nothing was audited by an independent replay. The held-out sets used for the final numbers were never used for a proof, and that is the one protection against the seed having tuned itself to its scoreboard, but a confirmation on sealed seeds with a predeclared gate is what the programme's rules require before any of this is a claim.

It is recursive in one sense and not in another. The seed's model and language direct its earning, its earning produces the experience the next model and language are fitted from, and the ledger shows the state improving the state across ten lives. But the operations that turn experience into a model, candidates into atoms, and proofs into keeps are fixed code. A later seed does not produce a better reflect or a better extend. The programme's definition of recursive growth, a later improver producing a better child improver, is not met here, and the result should not be described as if it were.

It is also bounded in ways that matter. The growth saturates: on `s1`'s last two mechanism lives neither a model nor an extension was kept, and the model gains fell from hundreds to single digits by the sixth life. The gain in the mechanism area is not uniform either: on the untouched held-out set, untrained family combinations fell from 24 to 18 tasks per world while trained worlds rose from 26 to 50, so the models the seed kept bought trained worlds partly at the expense of combinations it never lived. That is the safety failure of C-earned in a new form, and the seed's proofs cannot see it, because which pool a world came from is a label. Half the invented atoms are behaviourally new and some of them are surely junk that costs scan budget until a prune removes them. Proving cost nearly twice living. And every area shares one task form and one catalog; the seed has not met a second domain.

8 Lessons from building it

1. The proof is the seed. Every design choice that mattered was about what counts as a kept change: strictly better on a fresh held-out set, non-inferior everywhere the state is shared, cost charged. With that rule in place, a weak candidate costs a proof and nothing else, so the seed can propose freely.

2. Propose everything, then take it apart. Proving the combination first and the parts only on failure halved the proofs in the common case and lost nothing, because a combination that wins is kept whole.

3. Invention needs recurrence. A candidate atom that fits one unsolved task is usually a coincidence; one that fits two is usually a missing atom. The first version accepted single fits and every extension it proposed lost.

4. Let the seed remove what it added. Pruning atoms that never earned their keep was kept three times out of three, with gains as large as a model's. Carrying forward proven successes includes dropping the unproven.

5. A model from 48 worlds can be worth keeping. The programme's rule that a prior fitted on 25 worlds is worse than none was read on handed sets at tight budgets; here, with the proof deciding, the first model was kept on every seed and every area.

6. Shared state needs shared proofs. The 10% loss in an untouched area was invisible to any single-area proof.

7. Score growth on sets the seed never saw. The per-life numbers the seed reads are within a few percent of the fresh ones here, but that is a fact to be measured each time, not assumed.

8. Build for the host. One stall and one crash in the record were the machine's shared memory, not the seed; retries with fewer workers and periodic stack dumps cost nothing and kept the runs honest.

9 Next

The seed's own direction is set by its ledger, and the work is on the seed.

- Confirmation. A predeclared gate on sealed seeds: after L lives in an area, the carried state beats the initial state on a fresh held-out set by a stated margin, with non-inferiority on every other area, two replications, a committed forecast, and an independent replay of the ledger. Until then nothing here is a claim.
- Proof cost. Proofs cost nearly twice living. Sequential proofs that stop early when a candidate is clearly behind, and smaller cross-area sets, should bring that under one without changing what is kept.
- Pools the seed cannot see. The untrained-combination loss in mechanism worlds needs a label-free guard; the hedge that reserves half the budget for the no-prior order is the natural place to look.
- More areas. The next areas with real difference are a second task form and a second alphabet in the same catalog, then the string DSL banks, which need the seed's solver to be one of several rather than the one it has.
- The next level. The seed improves its state and not its operations. The first operation it could plausibly improve is the proof itself, learning from its ledger how large a held-out set a candidate of each kind needs, which is a trust rule earned from experience rather than written down. That is the level the programme calls L2, and the seed's ledger is the first data a learner of it could be fitted on.

10 Reproducibility

The repository is the RSI project at `projects/rsi`. The seed is `rsi/seed.py` (state, areas, the four operations, the proof) with `rsi/seed_areas.py` for metric worlds, `rsi/invention.py` for the grammar and the recurrence test, and `rsi/self_world_model.py` for the model. `seed_run.py--names1--seed644101--areamech--lives4` runs lives and resumes from the saved state; `seed_check.py--names1--areasmech,drawn7,metric777001` scores a seed on held-out sets no proof used and labels its inventions. States are in `seeds/<name>/state.json` and are committed after every run; the ledger inside each is the full record of every candidate, its proof and its cost. Logs are `runs_seed_*.log` and the checks `analysis/seed-check-<name>.json`.

Settings: 48 worlds of 100 tasks per life, budget 1,024 scans per task, 24 worlds per pool for an area's own held-out set and 12 for the cross-area check, tolerance 2%, group counts 6 to 8 and 2 to 4 groups per world by held-out likelihood, extension cap 40 atoms per life with a recurrence of 2, pruning after 2 lives, tie-break 77. Area specs are stored in the state so a proof can rebuild any area the seed has lived in. The environment variable `RSI_WORKERS` caps the worker pool; on this machine 6 or fewer is safe beside other jobs.

Seeds: `s1` on 644101 (versions 1 to 24 reported here), `s2` on 643101 and `s3` on 645101. The companion paper on the world and self model is `SELF-SIMULATION-WORLD-MODEL-2026-10-10.md`, the distillation of earlier growth results is `GROWTH-DISTILLATION-2026-10-10.md`, and a living copy of this paper with interactive figures is at `https://claude.ai/code/artifact/ebd48ba7-84b1-4bb2-b2e5-eb18375651bc`.